

CSCI-GA.1170-003 Fundamental Algorithms

Problem Set 4

Varun Sahni
N10338688
NetID: vs3606

October 7, 2026

Problem 4-1 (Heaps: Beyond the Maximum)

(a) Third largest key

The largest key is at the root. The larger root child is the second largest. The third largest is the maximum of the other root child and the existing children of the second largest.

```

1 FIND-3RD-MAX(S):
2   heapSize ← |S|
3   if heapSize < 3:
4     error "The heap must contain at least three keys"
5   if S[2] > S[3]:
6     secondIndex ← 2
7     otherIndex ← 3
8   else:
9     secondIndex ← 3
10    otherIndex ← 2
11    thirdLargest ← S[otherIndex]
12    leftChildIndex ← Left(secondIndex)
13    rightChildIndex ← Right(secondIndex)
14    if leftChildIndex ≤ heapSize:
15      thirdLargest ← max(thirdLargest, S[leftChildIndex])
16    if rightChildIndex ≤ heapSize:
17      thirdLargest ← max(thirdLargest, S[rightChildIndex])
18    return thirdLargest

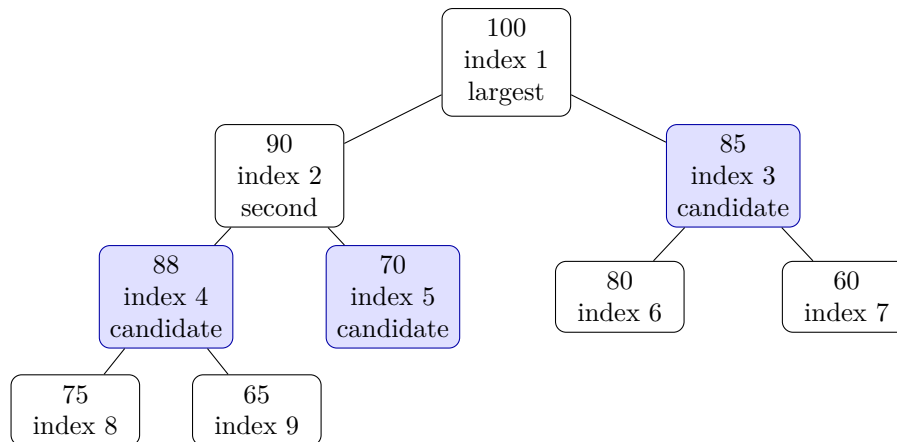
```

Here $\text{LEFT}(i) = 2i$ and $\text{RIGHT}(i) = 2i + 1$ return *indices*. A child exists exactly when its index is at most heapSize . Read $S[\text{leftChildIndex}]$ or $S[\text{rightChildIndex}]$ only after its corresponding existence check succeeds.

Correctness. Every descendant is smaller than its ancestors. Hence the larger root child is second largest. After excluding the root and that child, all remaining nodes belong to the subtree of the other root child or to one of the second largest's child subtrees. Each subtree's maximum is its root. Taking the maximum of these at most three candidate roots therefore returns the third largest. The heap is unchanged.

Time and space. One comparison identifies the second largest, and at most two more compare candidate keys. All other work is constant. Time is $\Theta(1)$ and auxiliary space is $O(1)$, versus $1 + 4 \log n$ for the given naive algorithm.

Example. In $S = (100, 90, 85, 88, 70, 80, 60, 75, 65)$, the second largest is 90 at index 2. The candidates are 85 at index 3, 88 at index 4, and 70 at index 5. The answer changes from 85 to 88, then stays 88.



The blue nodes are the only candidates. Each other remaining node is below a candidate and is smaller than it.

(b) Replace-Max

The naive algorithm removes the maximum and inserts the replacement. The intelligent algorithm replaces the root directly, then repairs downward.

```

1 REPLACE-MAX-NAIVE(S, replacementKey):
2   if |S| == 0: error "The heap is empty"
3   originalMaximum <- Extract-Max(S)
4   Insert(S, replacementKey)
5   return originalMaximum
6
7 REPLACE-MAX-INTELLIGENT(S, replacementKey):
8   if |S| == 0: error "The heap is empty"
9   originalMaximum <- S[1]
10  S[1] <- replacementKey
11  Max-Heapify(S, 1)
12  return originalMaximum
  
```

Correctness. Extraction and insertion keep a valid heap and replace exactly the original maximum. In the intelligent version, the child subtrees stay valid when the root changes. Only the root's comparisons with its children can fail, so max-heapify restores heap order. Both algorithms return the saved original maximum and keep every other key.

Time. Under the given operation costs, the naive version costs $2 \log n + O(1)$ and the intelligent version costs $\log n + O(1)$: two heap repairs versus one. Both have $O(\log(n+1))$ worst-case time. The intelligent version takes $\Theta(1)$ if the new root already dominates its children. For $n = 1$, both take constant time.

Space. Both use $O(1)$ auxiliary space with iterative heap operations. A recursive implementation of heapify uses $O(\log(n+1))$ stack space.

(c) Report-Greater

Naive algorithm. Repeatedly extract the maximum while it exceeds the threshold. Save the extracted keys, then reinsert them to restore the queue.

```

1 REPORT-GREATER-NAIVE(S, threshold):
2   reportedKeys <- empty list
3   remainingCount <- |S|
4   while remainingCount > 0:
5     if Maximum(S) <= threshold:
6       break
7     append Extract-Max(S) to reportedKeys
8     remainingCount <- remainingCount - 1
9   for each reportedKey in reportedKeys:
10    Insert(S, reportedKey)
11  return reportedKeys

```

Correctness. After each extraction, the list contains exactly the removed largest keys, all greater than the threshold. On stopping, the heap is empty or its maximum is at most the threshold, so every required key has been reported. Reinsertion restores the original keys. As a priority queue, S is unchanged; its internal array order may differ.

Time and space. For m reported keys there are m extractions, m insertions and at most $m+1$ maximum tests. The given primitive costs give $\Theta(1 + m \log(n+1))$ time. This also accounts for shrinking heap sizes: if $m \leq n/2$, the extraction sizes are at least $n/2$; if $m > n/2$, at least $\lfloor n/2 \rfloor$ extractions have that size and $m = \Theta(n)$. The saved output uses $O(m)$ space; iterative primitives need $O(1)$ additional space.

Intelligent algorithm. Start at the root. Report a key greater than the threshold and visit its children. Otherwise skip the entire subtree, since its descendants are no larger.

```

1 REPORT-GREATER-INTELLIGENT(S, threshold):
2   reportedKeys <- empty list
3   VISIT(S, 1, threshold, reportedKeys)
4   return reportedKeys
5
6 VISIT(S, nodeIndex, threshold, reportedKeys):
7   if nodeIndex > |S|:
8     return
9   if S[nodeIndex] <= threshold:
10    return
11  append S[nodeIndex] to reportedKeys
12  VISIT(S, Left(nodeIndex), threshold, reportedKeys)
13  VISIT(S, Right(nodeIndex), threshold, reportedKeys)

```

Correctness. Induct on subtree height. A missing node contributes nothing. A root at most the threshold has no qualifying descendants, so pruning is correct. Otherwise the root qualifies, and the recursive calls report exactly the qualifying keys in its two disjoint child subtrees by induction. Each key is reported once. The algorithm never writes to S , so even its array order is unchanged.

Time and space. Each reported node creates two child calls; the initial root call adds one. Thus there are exactly $2m+1$ calls, each with constant work, giving $\Theta(m+1)$ time independent of n . The output uses $O(m)$ space. The recursion stack uses $O(1 + \min\{m, \log(n+1)\})$ auxiliary space.

(d) Decrease-Key

The naive algorithm replaces the selected key by $+\infty$, extracts it, then inserts the decreased value. Here $+\infty$ is a temporary key larger than every valid key. The intelligent algorithm writes the decreased value and repairs downward.

```
1 DECREASE-KEY-NAIVE(S, keyIndex, decreasedKey):  
2   Increase-Key(S, keyIndex, +infinity)  
3   Extract-Max(S)           // discard the sentinel  
4   Insert(S, decreasedKey)  
5  
6 DECREASE-KEY-INTELLIGENT(S, keyIndex, decreasedKey):  
7   S[keyIndex] <- decreasedKey  
8   Max-Heapify(S, keyIndex)
```

Both procedures assume a valid index and $\text{decreasedKey} < S[\text{keyIndex}]$ before the change.

Correctness. The naive version removes exactly the selected original key: increase-key moves its sentinel upward, and extract-max removes that sentinel. Insertion then puts the decreased key into a valid heap. The intelligent version preserves the parent inequality, since decreasing a key cannot make it exceed its parent. Its child subtrees remain valid, so max-heapify repairs the only possible violations.

Time and space. The naive version costs $3 \log n + O(1)$ in the stated model; the intelligent version costs at most $\log n + O(1)$. Both have $O(\log(n+1))$ worst-case time and $O(1)$ auxiliary space with iterative primitives. More precisely, intelligent repair costs $O(h+1)$ for a node of height h , so a leaf takes $\Theta(1)$. Recursive heapify uses $O(h+1)$ stack space.

Why not use heapify for increase-key? Increasing a key can make it larger than its parent. Heapify compares only with children, so it cannot repair that upward violation. Increase-key must move the key upward instead.

Problem 4-2 (Quicksort vs Heapsort on Nice Arrays)

For distinct keys, c -niceness means $A[i] < A[j]$ whenever $j - i \geq c$. Ranks count from 1 for the smallest element. We take c to be a fixed positive integer.

(a) Rank bounds

The rank of $A[i]$ is its position in the fully sorted array: rank 1 is the smallest key, rank 2 the second smallest, and so on. Thus

$$\text{rank}(A[i]) = 1 + \#\{\text{keys smaller than } A[i]\}.$$

We use c -niceness to count keys that must be smaller or larger.

Lower bound. Positions $1, \dots, i - c$ are at least c positions to the left of i , so their keys must be smaller than $A[i]$. There are $\max\{0, i - c\}$ such keys. Adding one gives rank at least $\max\{1, i - c + 1\}$.

Upper bound. Positions $i + c, \dots, n$ must contain larger keys. There are $\max\{0, n - i - c + 1\}$ of them, so the rank is at most $n - \max\{0, n - i - c + 1\} = \min\{n, i + c - 1\}$. Therefore

$$\boxed{\max\{1, i - c + 1\} \leq \text{rank}(A[i]) \leq \min\{n, i + c - 1\}}.$$

To prove that these are the best bounds, start with the sorted array $(1, 2, \dots, n)$, where each value equals its rank. To reach the lower bound, reverse the block from position $\ell = \max\{1, i - c + 1\}$ to position i . This puts value ℓ at position i , giving rank ℓ . To reach the upper bound, start again with the sorted array and reverse the block from position i to position $u = \min\{n, i + c - 1\}$. This puts value u at position i , giving rank u .

Each reversed block has at most c entries. The only pairs put out of order are inside that block, so their positions differ by less than c . Both arrays therefore remain c -nice, and neither bound can be improved.

Now assume the pivot is the first entry $A[p]$ of each current subarray $A[p..r]$. After partitioning, it is at index q ; quicksort continues on $A[p..q - 1]$ and $A[q + 1..r]$.

(b) Both subarrays remain c -nice

Consider the first partition of $A[1..n]$, with pivot $x = A[1]$ and final pivot index q . We must show that *within each resulting part*, any two entries at least c positions apart are in increasing order.

Where swaps occur. By (a), the first entry has rank at most c , so $q \leq c$. Also, original positions $c + 1, \dots, n$ contain keys larger than x . Thus the scan encounters smaller keys only in positions $2..c$, and every swap stays within the first c positions. Entries after position c are unchanged.

Left part $A[1..q - 1]$. It has $q - 1 \leq c - 1$ entries. No pair of its indices is c positions apart, so it satisfies the definition of c -niceness automatically.

Right part $A[q + 1..n]$. In the scan, a key larger than x can only stay in place or move right when exchanged with a smaller key. The final pivot swap touches positions 1 and q , outside the right part.

Take any final right-part positions $u < v$ with $v - u \geq c$. Since $u \geq q + 1 \geq 2$, we have $v > c$, so the key at v is unchanged. The key now at u came from an original position $a \leq u$. Therefore

$v - a \geq v - u \geq c$. Original c -niceness gives

$$A_{\text{after}}[u] = A_{\text{before}}[a] < A_{\text{before}}[v] = A_{\text{after}}[v].$$

This proves the required order for every such pair in the right part. Hence both resulting subarrays are c -nice. Reindexing a subarray leaves index distances unchanged, so the same argument applies at later recursive calls.

A partition scans $n-1$ entries, so it takes $\Theta(n)$ time and $O(1)$ auxiliary space. The full quicksort analysis follows in (c).

(c) Best-case running time

By (a), a first-position pivot has rank at most c . Therefore its left part has size $s \leq c-1$, and its right part has size $n-s-1 \geq n-c$. By (b), both children are still c -nice. For any run, its time recurrence at a node is

$$T(A) = T(A_L) + T(A_R) + \Theta(n), \quad |A_L| = s \leq c-1, \quad |A_R| = n-s-1.$$

Let $B_c(n)$ denote the minimum runtime over all c -nice arrays of length n . Each child takes at least the best-case time for its size. Therefore

$$B_c(n) \geq \min_{0 \leq s \leq \min\{c-1, n-1\}} \{B_c(s) + B_c(n-s-1)\} + \alpha(n-1),$$

for a fixed $\alpha > 0$ accounting for the scan comparisons, with constant base costs. This is a lower bound because the two child minima need not occur together.

Follow the right child after each partition. After j partitions its size is at least $n-jc$. Let $d = \lfloor n/(2c) \rfloor$. Each of the first d partitions handles at least $n/2$ entries, so

$$B_c(n) \geq \alpha \sum_{j=0}^{d-1} (n-jc-1) = \alpha \left(d(n-1) - \frac{cd(d-1)}{2} \right) = \Omega(n^2/c).$$

For fixed c , this is $\Omega(n^2)$.

Quicksort is $O(n^2)$ on every array: at depth d its active subarrays are disjoint, their total size is at most n , and depth is at most n . A sorted array is also c -nice and has $T(n) = T(n-1) + \Theta(n) = \Theta(n^2)$, giving the same quadratic bound. Therefore

$$\boxed{B_c(n) = \Theta(n^2) \quad \text{for fixed } c.}$$

The constants may depend on c . Standard recursive quicksort uses $\Theta(n)$ stack space for fixed c : the right-child chain loses at most c entries per call, so it has $\Omega(n/c)$ depth; no chain exceeds n . Each partition uses $O(1)$ extra space.

(d) Sort-Nice

Use a min-heap containing at most b candidates. Build it from the first b entries. Repeatedly output its minimum and insert the next unread entry, if one remains.

Min-heap procedures. Use the CLRS heap procedures with comparisons reversed. The array H has capacity $H.length$ and active size $H.heapSize$. Its parent and child indices are

$$\text{PARENT}(i) = \lfloor i/2 \rfloor, \quad \text{LEFT}(i) = 2i, \quad \text{RIGHT}(i) = 2i + 1.$$

Min-heap order means each parent key is at most its children's keys. Therefore $H[1]$ is the minimum.

```

1 MIN-HEAPIFY(H, nodeIndex):
2   leftIndex <- Left(nodeIndex)
3   rightIndex <- Right(nodeIndex)
4   smallestIndex <- nodeIndex
5   if leftIndex <= H.heapSize:
6     if H[leftIndex] < H[smallestIndex]:
7       smallestIndex <- leftIndex
8   if rightIndex <= H.heapSize:
9     if H[rightIndex] < H[smallestIndex]:
10      smallestIndex <- rightIndex
11   if smallestIndex != nodeIndex:
12     exchange H[nodeIndex] and H[smallestIndex]
13     MIN-HEAPIFY(H, smallestIndex)
14
15 BUILD-MIN-HEAP(H):
16   H.heapSize <- H.length
17   for nodeIndex <- floor(H.length / 2) downto 1:
18     MIN-HEAPIFY(H, nodeIndex)

```

Heapify correctness. Assume both child subtrees are min-heaps. If the current key is smallest, the subtree is already valid. Otherwise swap it with the smaller child. The new root is no larger than either child; only the chosen child's subtree may now need repair. Induction on height proves the recursive call restores that subtree. Each call moves down one level, so time and stack space are $O(\log(m+1))$ for an m -entry heap.

Build correctness and time. Before each iteration, every higher-indexed node roots a min-heap. Initially these nodes are leaves. Since children have larger indices than their parent, heapify's assumption holds and the iteration establishes the invariant for the next node. At termination, the root is a min-heap. Nodes of height h number at most $\lceil m/2^{h+1} \rceil$; summing their $O(h+1)$ repair costs gives $O(m)$, since $\sum_{h \geq 0} (h+1)/2^h$ is constant (the rounding terms add $O(\log^2(m+1)) = O(m)$). Thus build time is $\Theta(m)$. It modifies the array in place and uses $O(\log(m+1))$ stack space for recursive heapify.

```

1  HEAP-EXTRACT-MIN(H):
2      if H.heapSize == 0:
3          error "The heap is empty"
4      minimumKey <- H[1]
5      H[1] <- H[H.heapSize]
6      H.heapSize <- H.heapSize - 1
7      if H.heapSize > 0:
8          MIN-HEAPIFY(H, 1)
9      return minimumKey

```

The root is the minimum. Replacing it by the last key and shrinking the active size leaves both child subtrees valid, so heapify restores min-heap order. This takes $O(\log(m+1))$ time and recursive stack space.

```

1  HEAP-DECREASE-KEY(H, keyIndex, newKey):
2      if newKey > H[keyIndex]:
3          error "The new key must not increase"
4      H[keyIndex] <- newKey
5      while keyIndex > 1:
6          parentIndex <- Parent(keyIndex)
7          if H[parentIndex] <= H[keyIndex]:
8              break
9          exchange H[keyIndex] and H[parentIndex]
10         keyIndex <- parentIndex
11
12  MIN-HEAP-INSERT(H, newKey):
13      H.heapSize <- H.heapSize + 1
14      H[H.heapSize] <- +infinity
15      HEAP-DECREASE-KEY(H, H.heapSize, newKey)

```

Decrease-key moves the changed key upward while it is smaller than its parent. Only that parent edge can violate heap order, and each swap moves the possible violation one level upward. It stops at the root or a valid parent edge. Insert adds a leaf with temporary key $+\infty$, then decreases it to the new key. Both take $O(\log(m+1))$ time and $O(1)$ extra space. In Sort-Nice, insertion always follows extraction, so the capacity b is sufficient.

Sort-Nice. Here $\text{windowSize} = b$ and $\text{arraySize} = n$. Write B for the returned output array. The input A stays unchanged, so later trials can reuse it.

```

1  SORT-NICE(A, windowSize):
2      arraySize <- |A|           // 1 <= windowSize <= arraySize
3      candidateHeap <- copy of A[1..windowSize]
4      BUILD-MIN-HEAP(candidateHeap)
5      sortedOutput <- new array of length arraySize
6      nextInputIndex <- windowSize + 1
7      for outputIndex <- 1 to arraySize:
8          minimumKey <- HEAP-EXTRACT-MIN(candidateHeap)
9          sortedOutput[outputIndex] <- minimumKey
10         if nextInputIndex <= arraySize:
11             MIN-HEAP-INSERT(candidateHeap, A[nextInputIndex])
12             nextInputIndex <- nextInputIndex + 1
13     return sortedOutput

```

Permutation guarantee for every input. Each original entry is inserted exactly once: the first b at construction and all remaining entries through nextInputIndex . At output position $t = \text{outputIndex}$, the heap contains exactly the inserted entries not yet output. It is nonempty until all n iterations finish. Each extraction removes one entry and appends that entry exactly

once to B . After n outputs nothing remains; B is a permutation, regardless of niceness or the choice of b .

Sortedness when $c \leq b$. Induct on output position t . Assume the first $t - 1$ outputs are the globally smallest $t - 1$ entries in order. Immediately before extraction t , all entries through original position $\min\{n, t + b - 1\}$ have been inserted. By (a), an entry of global rank t has original position

$$i \leq t + c - 1 \leq t + b - 1.$$

It has therefore been inserted, and it has not yet been output. It is in the heap and is the smallest remaining original entry. Extract-min returns it. The initial case $t = 1$ follows from the same inequality; induction proves the output sorted.

Running time. Building costs $O(b)$. There are n extractions and $n - b$ insertions on heaps of size at most b , plus $O(n)$ output/bookkeeping:

$$T(n, b) = O(b + n + (2n - b) \log(b + 1)) = \boxed{O(n \log(b + 1))}.$$

For $b \geq 2$ this is the requested $O(n \log b)$; for $b = 1$ it is $\Theta(n)$. Heap space is $O(b)$, with $O(n)$ output space.

(e) Doubling guesses

Use $b_i = \min\{n, 2^i\}$ to respect (d)'s allowed range, with $i = 0, 1, \dots$. Every trial uses the original A . Checking whether the trial output is sorted costs $\Theta(n)$ in the worst case. The last trial certainly succeeds at $b = n$, because then all entries are in the heap before any extraction.

If A is c^* -nice, put $\hat{c} = \min\{c^*, n\}$ and $L = \lceil \log_2 \hat{c} \rceil$. A trial succeeds no later than $i = L$, since its capacity is at least $\hat{c}$. Each trial costs $O(n(1 + i))$, including the sortedness scan, so

$$\begin{aligned} T(n, c^*) &\leq O\left(n \sum_{i=0}^L (1 + i)\right) \\ &= O\left(n \left((L + 1) + \frac{L(L + 1)}{2}\right)\right) \\ &= \boxed{O(n(1 + \log^2 c^*))}. \end{aligned}$$

For $c^* \geq 2$ this is $O(n \log^2 c^*)$; for $c^* = 1$ the first trial returns the already-sorted input in $\Theta(n)$. Earlier success only improves this bound. Space is $O(n)$ when each unsuccessful output is discarded before the next trial.

Reversing an array of size $n = 2^L$ gives a family with minimum niceness parameter $c^* = n$. Each trial with $b < n$ outputs $n - b + 1$ first and later outputs the unseen value 1, so fails. For powers $b \leq n/2$, the remaining unseen descending values are successive new minima and are inserted into a heap of size b , requiring $\Theta(\log b)$ upward work per insertion. There are at least $n/2$ such insertions per trial. Hence the trials cost $\Omega(n \sum_{i=1}^{L-1} i) = \Omega(nL^2)$ in total. Thus this schedule can require $\Theta(n \log^2 c^*)$ time.

(f) Faster guesses

After testing $b = 1$, use guesses $2, 4, 16, 256, \dots$, squaring the previous guess and capping at n .

```

1 NICE-BUT-WEIRD-SORT-FAST(A):
2   arraySize <- |A|           // requires arraySize >= 1
3   windowSize <- 1
4   while true:
5     sortedOutput <- SORT-NICE(A, windowSize)
6     if IS-SORTED(sortedOutput):
7       return sortedOutput
8     if windowSize == 1:
9       windowSize <- min(arraySize, 2)
10    else:
11      windowSize <- min(arraySize, windowSize * windowSize)

```

Correctness and termination. Every trial output is a permutation. The algorithm returns only a sorted one. A failed trial with $b < n$ increases b strictly; a trial at $b = n$ always succeeds. Thus termination and correctness follow without knowing c^* .

For $c^* > 1$, if the successful trial is not the first $b = 2$ trial, its previous guess was below $\hat{c}$: otherwise (d) would already have guaranteed success. The last capacity is therefore at most $\hat{c}^2$ (or the smaller cap n), so its logarithm is less than $2 \log_2 \hat{c}$. Before capping, the positive logarithms are $1, 2, 4, 8, \dots$. Their sum is at most twice the largest one. The number of sortedness checks is also at most a constant plus this sum. Consequently

$$T(n, c^*) = O(n(1 + \log c^*)).$$

For $c^* \geq 2$, this is the requested $O(n \log c^*)$; for $c^* = 1$, it is $\Theta(n)$. Space is $O(n)$, discarding each failed trial before the next.

Problem 4-3 (Assigning Letter Grades)

A score s belongs to band j exactly when $c_{j-1} < s \leq c_j$. The finite cutoffs $c_1 < \dots < c_{k-1}$ are already ordered. Sorting inside a band is unnecessary.

(a) Randomized quicksort and Partition-Value

Randomized quicksort returns a sorted permutation by the partition postcondition and induction on subarray length. In sorted order, a higher band cannot precede a lower band: if s belongs to band a and t to band $b > a$, then $s \leq c_a \leq c_{b-1} < t$. Therefore sorting solves the grouping problem.

Lecture randomized quicksort has expected time $O(n \log n)$, so Hari's algorithm has that bound. It uses $O(n)$ worst-case recursion-stack space in the standard implementation.

Partition-Value. Since c need not be an entry, scan *all* entries, do not reserve a pivot slot, and do not perform a final pivot exchange.

```

1 PARTITION-VALUE(A, startIndex, endIndex, cutoffValue):
2   lowerEnd <- startIndex - 1
3   for scanIndex <- startIndex to endIndex:
4     if A[scanIndex] <= cutoffValue:
5       lowerEnd <- lowerEnd + 1
6       exchange A[lowerEnd] and A[scanIndex]
7   return lowerEnd

```

Invariant. Write $p = \text{startIndex}$, $r = \text{endIndex}$, $q = \text{lowerEnd}$, $t = \text{scanIndex}$ and $c = \text{cutoffValue}$. Before iteration t , $A[p..q] \leq c$, $A[q+1..t-1] > c$, and $A[t..r]$ is unprocessed. The subarray remains a permutation of its original entries. Initially both processed regions are empty. If the next entry is at most c , incrementing q and exchanging moves it into the low region and the displaced high-region entry (if any) to the end of the high region. Otherwise it simply extends the high region. Thus every branch maintains the invariant. At termination $t = r + 1$, it gives

$$A[p..q] \leq c < A[q+1..r].$$

Exactly $r - p + 1$ entries are scanned; time is $\Theta(r - p + 1)$ for a nonempty interval and auxiliary space $O(1)$. If none qualify, $q = p - 1$; if all qualify, $q = r$. An empty interval returns $p - 1$ in constant time. Equality belongs on the left, as the band definition requires.

(b) Mira's sequential cutoffs

```

1 SEQUENTIAL-BAND-SORT(A, cutoffs[1..bandCount-1]):
2   remainingStart <- 1
3   for cutoffIndex <- 1 to bandCount-1:
4     if remainingStart > |A|:
5       return
6     bandEnd <- PARTITION-VALUE(
7       A, remainingStart, |A|, cutoffs[cutoffIndex])
8     remainingStart <- bandEnd + 1

```

Correctness. Before cutoff c_j , the completed prefix consists of bands $1, \dots, j-1$ in order, and every remaining score exceeds c_{j-1} . The first step has empty prefix and lower bound $-\infty$. Partitioning the suffix at c_j places exactly the scores in $(c_{j-1}, c_j]$, namely band j , immediately after that prefix. The new suffix exceeds c_j . At the end only band k remains, so all bands are

grouped. Swaps preserve all scores. Empty bands create an empty block without invalidating the invariant.

Each of $k - 1$ partitions scans at most n entries, with constant loop overhead, so worst-case time is $O(nk)$ for $k \geq 2$; $k = 1$ needs no rearrangement and takes $O(1)$ for this procedure. Auxiliary space is $O(1)$.

Tight example. Let $c_j = j$ for $j = 1, \dots, k - 1$ and $A[t] = k + t$ for $t = 1, \dots, n$. All scores lie in band k . Every partition returns `lowerEnd` = 0 and leaves `remainingStart` = 1, hence scans all n scores. The total is

$$(k - 1)\Theta(n) = \boxed{\Theta(nk)} \quad (k \geq 2).$$

(c) Band-Sort

Cutoffs are stored in the shared read-only array `cutoffs[1..k - 1]`, with `cutoffs[j] = c_j`. The parameters `firstCutoff = i` and `lastCutoff = j` specify the remaining cutoff indices; `startIndex = p` and `endIndex = r` specify the score interval. A one-score interval may still need classification, so return only for an empty score interval or no remaining cutoffs.

```

1 BAND-SORT(A, startIndex, endIndex, firstCutoff, lastCutoff):
2   if startIndex > endIndex or firstCutoff > lastCutoff:
3     return
4   middleCutoff <- floor((firstCutoff + lastCutoff) / 2)
5   lowerEnd <- PARTITION-VALUE(
6     A, startIndex, endIndex, cutoffs[middleCutoff])
7   BAND-SORT(A, startIndex, lowerEnd,
8     firstCutoff, middleCutoff-1)
9   BAND-SORT(A, lowerEnd+1, endIndex,
10    middleCutoff+1, lastCutoff)
```

Initial call:

$$\boxed{\text{BAND-SORT}(A, 1, n, 1, k - 1)}.$$

Here $q = \text{lowerEnd}$ and $m = \text{middleCutoff}$. The left child includes q : unlike quicksort, no actual pivot element is removed from the array.

Inductive claim. If every score in $A[p..r]$ lies in $(c_{i-1}, c_{j+1}]$, the procedure preserves its multiset and groups it into bands $i, i + 1, \dots, j + 1$ in ascending band order. This range invariant is true for the initial call since $c_0 = -\infty$ and $c_k = +\infty$. Induct on the number $t = j - i + 1$ of remaining cutoffs.

Base cases. An empty score interval is already grouped. If there are no cutoffs, $i = j + 1$; the range invariant puts all scores in the single band i , so returning is correct.

Induction step. With $t \geq 1$ and a nonempty interval, value partition at c_m preserves all scores and yields a left part in $(c_{i-1}, c_m]$ and a right part in $(c_m, c_{j+1}]$. The left recursive call uses fewer cutoffs $c_i, \dots, c_{m-1}$ and satisfies its range precondition; the right call uses fewer cutoffs $c_{m+1}, \dots, c_j$ and satisfies its own. By induction they group bands $i, \dots, m$ and $m + 1, \dots, j + 1$, respectively. Every band on the left precedes every band on the right, and their concatenation is the original interval rearranged in the required order. This proves the claim. The number of remaining cutoffs strictly decreases on each nonbase recursive call, so the procedure terminates even when one score part is empty.

(d) Recursion depth, disjointness and runtime

A node with t cutoffs gives each child at most $\lfloor t/2 \rfloor$ cutoffs: the middle cutoff is removed and the two remaining cutoff counts differ by at most one. Starting with $t = k - 1$, a node at depth d has at most

$$\left\lfloor \frac{k-1}{2^d} \right\rfloor$$

cutoffs. For $D = \lceil \log_2 k \rceil$, $2^D \geq k > k - 1$, so no partition is performed at depth D . There are at most D partition levels, and the recursion's edge depth, including base calls, is at most D . For $k = 1$, this is zero.

Disjointness. The root is the only interval at depth zero. Each interval $[p, r]$ divides into disjoint children $[p, q]$ and $[q + 1, r]$ whose union is its parent (empty intervals are allowed). Children of disjoint parents cannot intersect. Induction on depth therefore proves disjointness at every level. Thus their total nonempty length is at most n .

At a partition level, scanning costs at most $O(n)$ in total. The constant overhead is also $O(n)$: there are at most n nonempty intervals at that level. Empty child calls add at most twice the number of internal nodes overall, so they do not change the bound. Consequently

$$T(n, k) = O(n \lceil \log_2 k \rceil + 1) = O(n \log k) \quad (k \geq 2).$$

For $k = 1$ the procedure returns immediately in $O(1)$. It uses $O(\log k)$ recursion-stack space and $O(1)$ extra space in each value partition; no separate output array is needed.

For $k = 2$, there is one scan and the runtime is $\Theta(n)$, matching Mira and improving on Hari's expected $O(n \log n)$. For $k = n \geq 2$, the guarantee is $O(n \log n)$ in the worst case, matching Hari's expected bound while improving on Mira's $O(n^2)$ worst-case bound.

(e) Smallest remaining cutoff

Choosing the smallest remaining cutoff c_i leaves no cutoffs for the left recursive call, so it returns immediately. Only the right call continues, with one fewer cutoff. There are at most $k - 1$ partitions, each taking $O(n)$ time, which gives an upper bound of $O(nk)$.

To show this bound is tight, let every score exceed c_{k-1} . Each partition then leaves the left part empty and all n scores in the right part. All $k - 1$ partitions therefore scan the entire array. With t remaining cutoffs, the running time on this input satisfies

$$W(n, t) = W(n, t - 1) + \Theta(n), \quad W(n, 0) = \Theta(1).$$

Thus the worst-case time is

$$W(n, k - 1) = \Theta(n(k - 1) + 1) = \Theta(nk) \quad (k \geq 2).$$

For $k = 1$, there are no cutoffs and the running time is $\Theta(1)$. The recursion stack uses $O(k)$ space and can have $\Theta(k)$ depth.

This resembles **Mira's sequential algorithm from (b)**: partition around c_1 , then partition the remaining suffix around c_2 , and continue in increasing cutoff order.

This is the same unbalanced-partition behavior as quicksort with an extreme pivot, whose recurrence is $T(n) = T(n - 1) + \Theta(n)$. Here one cutoff is removed per level while all scores may stay in the right part; in quicksort one pivot is removed per level. For $k = n$, both have quadratic worst-case running time.